Nome: Ana Carolina dos Santos
Matrícula: 927633

Parte 2:

Operação	Retorno	Pilha(base -> topo)	Topo
empilha(10)	-	10	1
empilha(20)	-	10, 20	2
desempilha()	20	10	1
empilha(30)	-	10, 30	2
empilha(40)	-	10, 30, 40	3
desempilha()	40	10, 30	2
desempilha()	30	10	1
empilha(50)	-	10, 50	2

Parte 4.1:

Passada	Carry Final	B(base -> topo)	R(base -> topo)
1	9	1, 7, 2, 5	9
2	7	2, 5, 1	9, 7
3	5	1, 2	9, 7, 5
4	2	1	9, 7, 5, 2
5	1	vazio	9, 7, 5, 2, 1

Parte 4.3:

a) Foram feitas 10 comparações. Na passada com r elementos restantes são feitas r − 1 comparações, logo o total é (n−1) + (n−2) + … + 1 + 0 = n(n−1)/2 (para n = 5: 5·4/2 = 10). É exatamente o mesmo número do Bubble Sort clássico sem otimização, que compara pares adjacentes em n−1, n−2, …, 1 passadas. As duas versões são $O(n^2)$ e fazem o mesmo número de comparações mesmo com o vetor já ordenado (no teste com {1, 2, 3, 4, 5} também foram 10), pois nenhuma tem parada antecipada.

b) O Bubble Sort com vetor ordena no próprio vetor, com memória extra O(1) (uma variável temporária). Com pilhas, cada uma das três pilhas (A, B e R) tem capacidade n, ou seja, cerca de 3n posições extras, além do vetor de entrada e do vetor de saída: memória extra O(n). (Em cada instante A + B guardam no máximo n−1 elementos e R guarda o resto, mas, com vetores de tamanho fixo, a alocação é de 3n.)

c) Perdemos o acesso direto por índice (v[i]) e, portanto, a troca de posições (swap entre elementos adjacentes). Na pilha só enxergamos o topo. O código compensa de duas formas: (1) a variável carry guarda a “bolha”, ou seja, o maior valor visto até o momento na passada; (2) os elementos são transferidos de A para B a cada passada, decidindo o que empilhar primeiro (x ou carry), e o maior é depositado em R. Como a transferência inverte a ordem, a troca de papéis entre A e B prepara a entrada da próxima passada.

d) Para ordenar um vetor comum, não vale a pena: o algoritmo continua $O(n^2)$, gasta $O(n)$ de memória extra e é mais lento por causa das operações de empilha/desempilha, quando o Bubble Sort com vetor já faz o serviço no lugar (e há algoritmos bem melhores, como Merge e Quick Sort). A ideia é útil quando os dados só podem ser acessados em modo LIFO (estrutura com acesso restrito ao topo, como uma pilha de execução ou de desfazer/refazer) ou em contextos didáticos para exercitar o TAD. Nesse caso, o desafio ordenaPilha (uma pilha auxiliar e uma variável) é uma solução mais enxuta, ainda $O(n^2)$ no pior caso, mas sem a terceira pilha.